

Decrease and Conquer

Three major variations of decrease and conquer

- Decrease by a Constant
- Decrease by a Constant factor
- Variable size Decrease

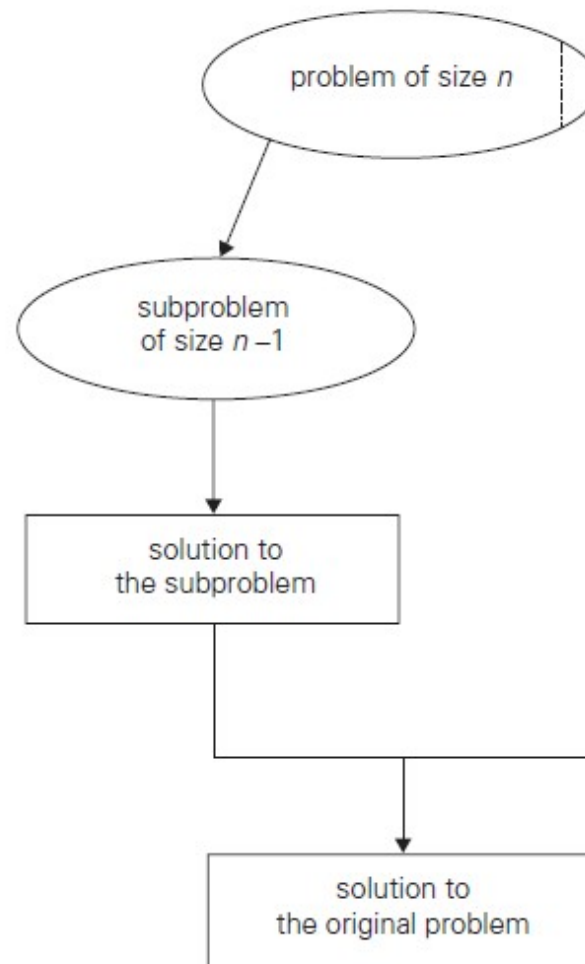


FIGURE 4.1 Decrease-(by one)-and-conquer technique.

$$f(n) = \begin{cases} f(n-1) \cdot a & \text{if } n > 0, \\ 1 & \text{if } n = 0, \end{cases}$$

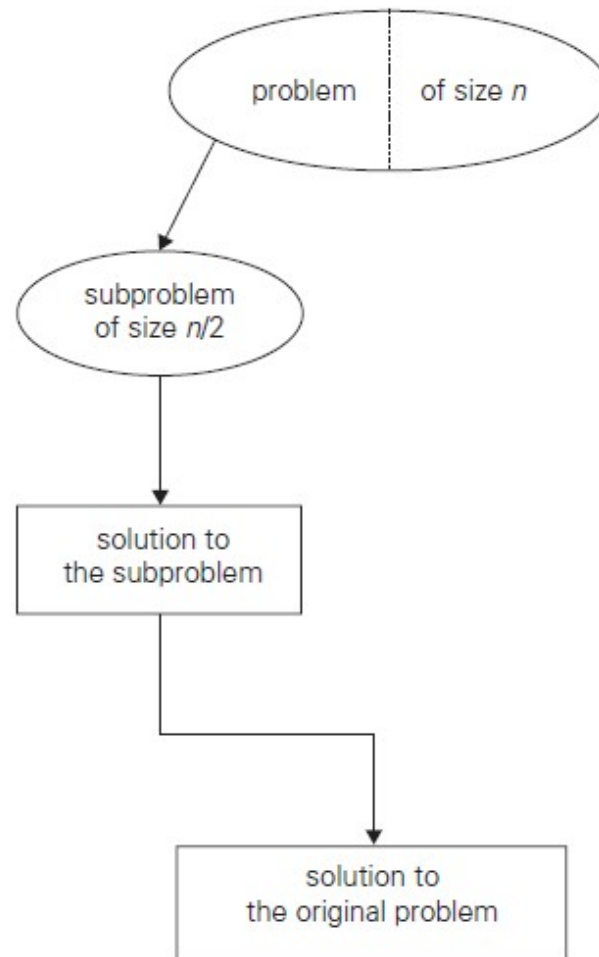


FIGURE 4.2 Decrease-(by half)-and-conquer technique.

$$a^n = \begin{cases} (a^{n/2})^2 & \text{if } n \text{ is even and positive,} \\ (a^{(n-1)/2})^2 \cdot a & \text{if } n \text{ is odd,} \\ 1 & \text{if } n = 0. \end{cases}$$

Topics Cover

- Insertion Sort
- Binary Search
- Topological Sorting

**Insertion Sort -
Already covered**

Insertion Sort

- Suppose we know how to insert a new element x in its proper place in an already sorted array A of size k , to get a new sorted array of size $k+1$
- Use this to sort the given array A of size n as follows:
 - Insert $A[1]$ in the sorted array $A[0]$. So now $A[0], A[1]$ are sorted
 - Insert $A[2]$ in the sorted array $A[0], A[1]$. So now $A[0], A[1], A[2]$ are sorted
 - Insert $A[3]$ in the sorted array $A[0], A[1], A[2]$. So now $A[0], A[1], A[2], A[3]$ are sorted
 -
 - Insert $A[i]$ in the sorted array $A[0], A[1], \dots, A[i-1]$. So now $A[0], A[1], \dots, A[i]$ are sorted
 - Continue until $i = n-1$ (outer loop)

Example of first step

A

5	7	11	13	20	22	
---	---	----	----	----	----	--

 Insert $x = 15$

Example of first step

A

5	7	11	13	20	22	
---	---	----	----	----	----	--

 Insert $x = 15$

Compare with 22. $x < 22$, so move 22 right

5	7	11	13	20	15	22
---	---	----	----	----	----	----

Example of first step

A

5	7	11	13	20	22	
---	---	----	----	----	----	--

 Insert $x = 15$

Compare with 22. $x < 22$, so move 22 right

5	7	11	13	20	15	22
---	---	----	----	----	----	----

Compare with 20. $x < 20$, so move 20 right

5	7	11	13	15	20	22
---	---	----	----	----	----	----

Example of step: Inserting an element in to sorted array

A

5	7	11	13	20	22	
---	---	----	----	----	----	--

 Insert $x = 15$

Compare with 22. $x < 22$, so move 22 right

5	7	11	13	20	15	22
---	---	----	----	----	----	----

Compare with 20. $x < 20$, so move 20 right

5	7	11	13	15	20	22
---	---	----	----	----	----	----

Compare with 13. $x > 13$, so stop

A

5	7	11	13	15	20	22
---	---	----	----	----	----	----

Sort using the insertion sort

A

7	5	13	11	22	20
---	---	----	----	----	----

Insert 5 in 7

5	7	13	11	22	20
---	---	----	----	----	----

Insert 13 in 5, 7

5	7	13	11	22	20
---	---	----	----	----	----

Insert 11 in 5, 7, 13

5	7	11	13	22	20
---	---	----	----	----	----

Insert 22 in 5, 7, 11, 13

5	7	11	13	22	20
---	---	----	----	----	----

Insert 20 in 5, 7, 11, 13, 22

5	7	11	13	20	22
---	---	----	----	----	----

Insertion Sort Code

```
void InsertionSort (int A[ ], int n)
{
    int i, j, item;
    for (i=1; i<n; i++)
    { /* Insert the element in A[i] */
        item = A[i] ;
        for (j = i-1; j >= 0; j--)
            if (item < A[j])
            { /* push elements down*/
                A[j+1] = A[j];
                A[j] = item ;    /* can do this once finally also */
            }
            else break; /*inserted, exit loop */
    }
}
```

Time Complexity $O(n^2)$

Insertion Sort Code

```
void InsertionSort (int A[ ], int n)
```

```
{
```

```
    int i, j, item;
```

```
    for (i=1; i<n; i++)
```

```
    { /* Insert the element in A[i] */
```

```
        item = A[i] ;
```

```
        for (j = i-1; j >= 0; j--)
```

```
            if (item < A[j])
```

```
            { /* push elements down*/
```

```
                A[j+1] = A[j];
```

```
                A[j] = item ;
```

```
            }
```

```
            else break; /*inserted, exit loop */
```

```
    }
```

```
}
```

Time Complexity $O(n^2)$

Steps

1

n

n-1

$\sum_1^{(n-1)} (i+1) = [n(n+1)/2] - 1$

$\sum_1^{(n-1)} i = (n-1)n/2$

$(n-1)n/2$

$(n-1)n/2$

Insertion Sort Analysis

Time Efficiency $C(n)$:

- Worst Case $C_{\text{worst}}(n) = \theta(n^2)$
- Average Case $C_{\text{avg}}(n) = \theta(n^2)$
- Best Case $C_{\text{best}}(n) = n-1 = \theta(n)$ (also fast on almost sorted arrays)

Binary Search

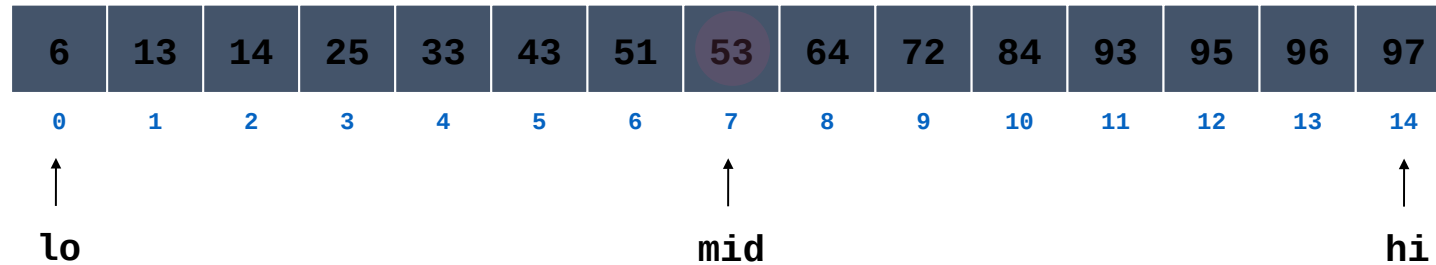
Binary Search

- Binary search. Given `value` and sorted array `a[]`, find index `i` such that `a[i] = value`, or report that no such index exists.
- Invariant. Algorithm maintains `a[lo] ≤ value ≤ a[hi]`
- Ex. Binary search for 33.

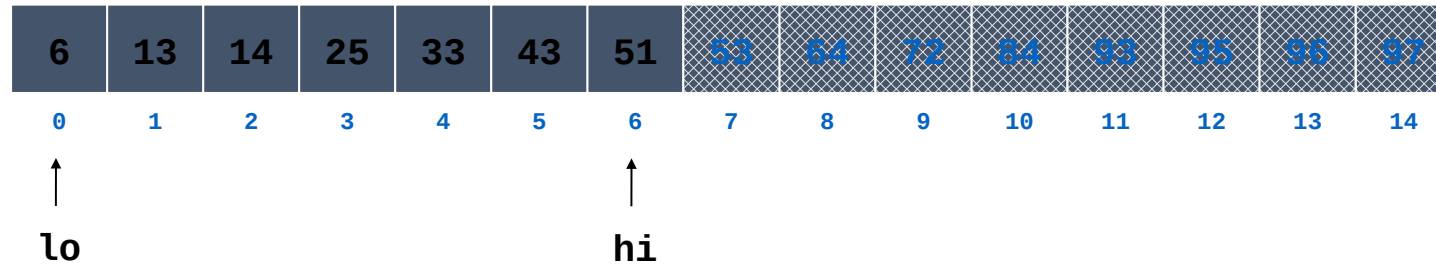
6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
↑ lo														↑ hi

Binary Search

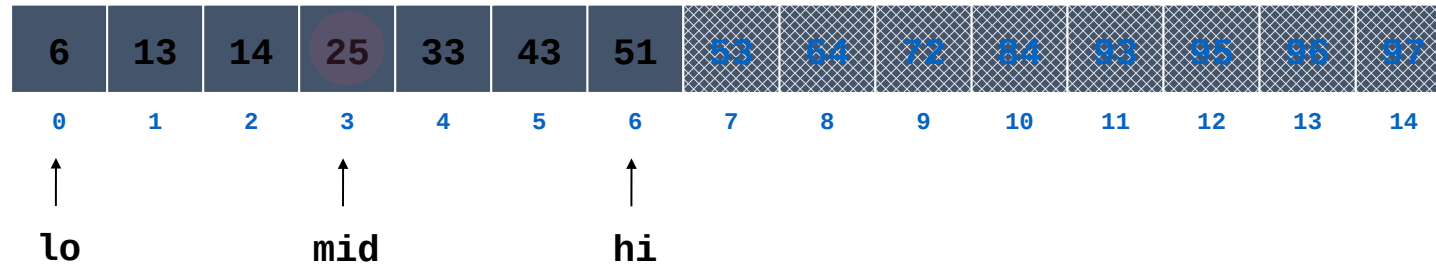
- Ex. Binary search for 33.



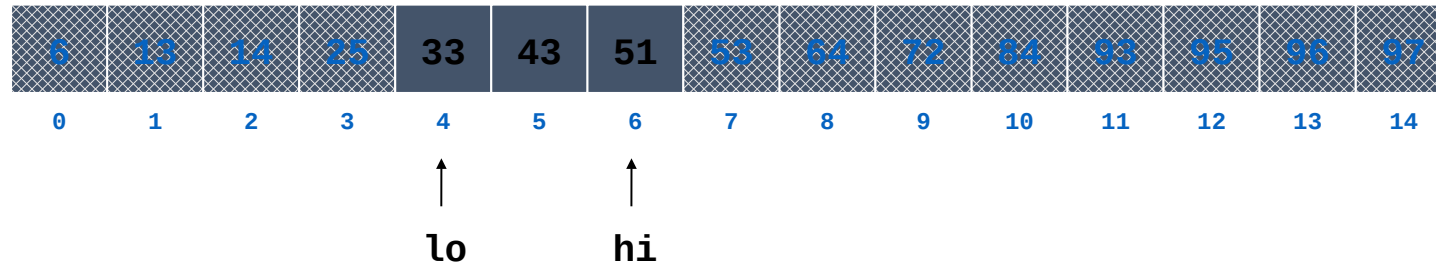
Binary Search



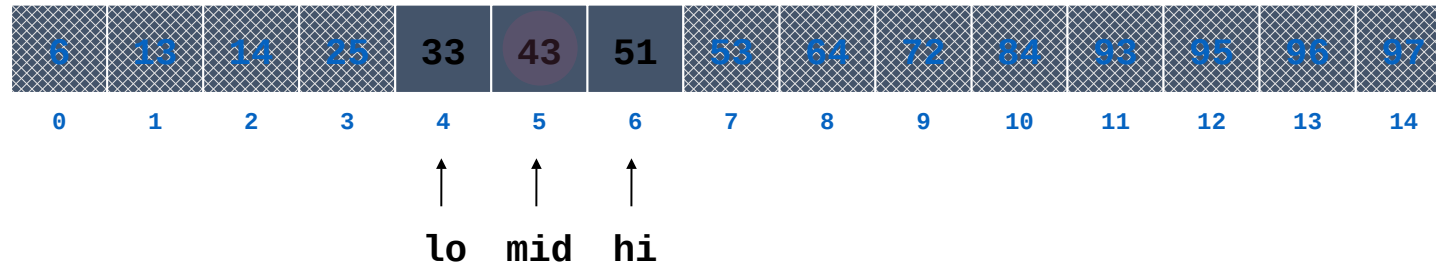
Binary Search



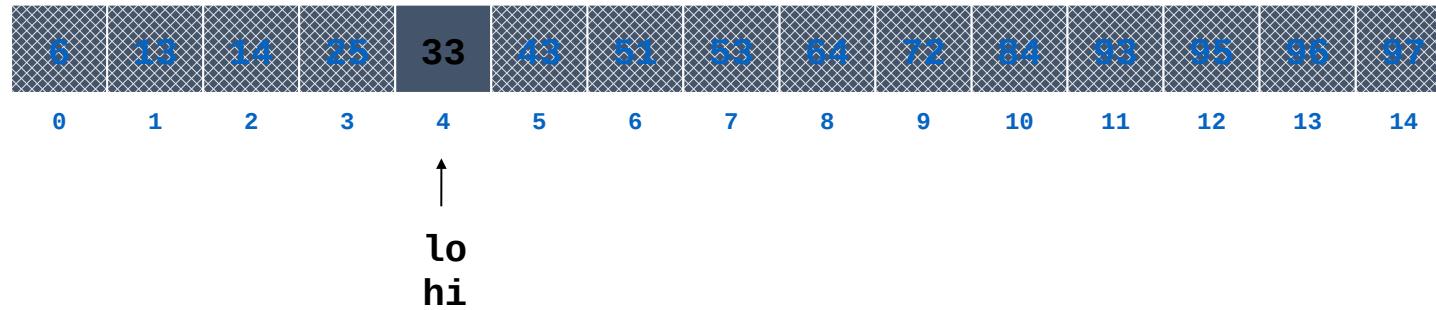
Binary Search



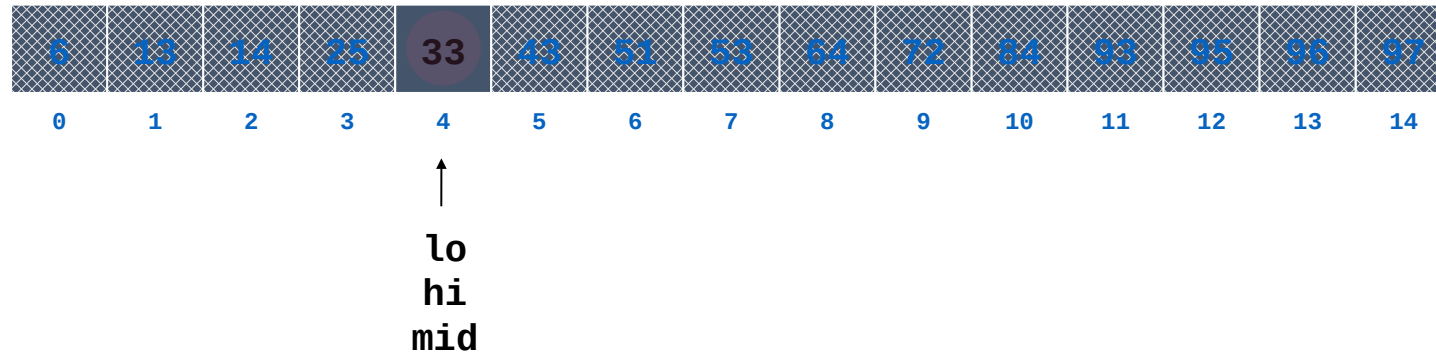
Binary Search



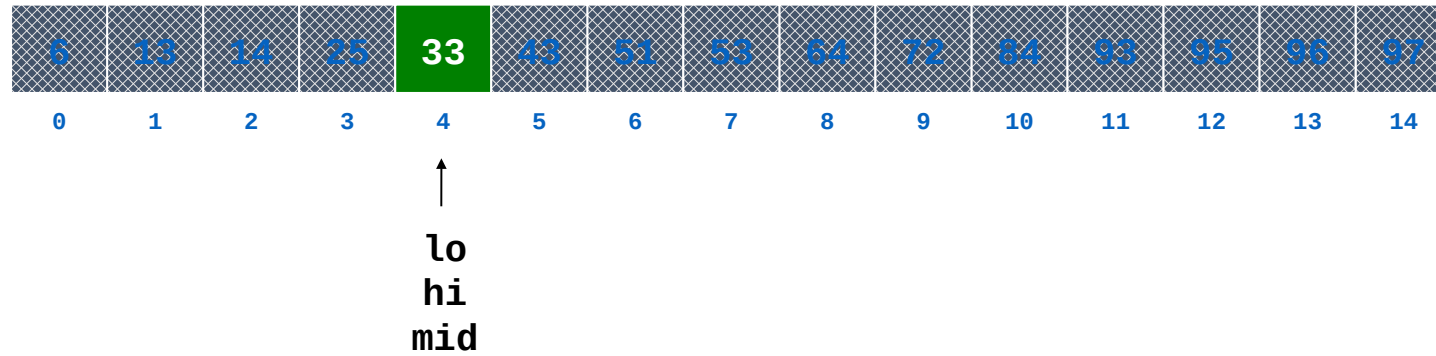
Binary Search



Binary Search



Binary Search



Binary Search Algorithm

ALGORITHM *BinarySearch*($A[0..n - 1]$, K)

//Implements nonrecursive binary search

//Input: An array $A[0..n - 1]$ sorted in ascending order and
// a search key K

//Output: An index of the array's element that is equal to K
// or -1 if there is no such element

$l \leftarrow 0$; $r \leftarrow n - 1$

while $l \leq r$ **do**

$m \leftarrow (l + r) / 2$

if $K = A[m]$ **return** m

else if $K < A[m]$ $r \leftarrow m - 1$

else $l \leftarrow m + 1$

return -1

Binary Search



```
int binarySearch(int a[], int n, const int x, int l, int r)
{
    if(l > r) return -1; // x not found

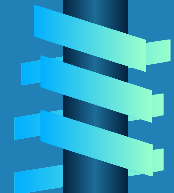
    int m = (l + r)/2;
    if (x == a[m]) return m;
    else if (x > a[m]) binarySearch(a, n, x, m+1, r);}
    else binarySearch(a, n, x, l, m-1);

}
```

Binary Search Complexity



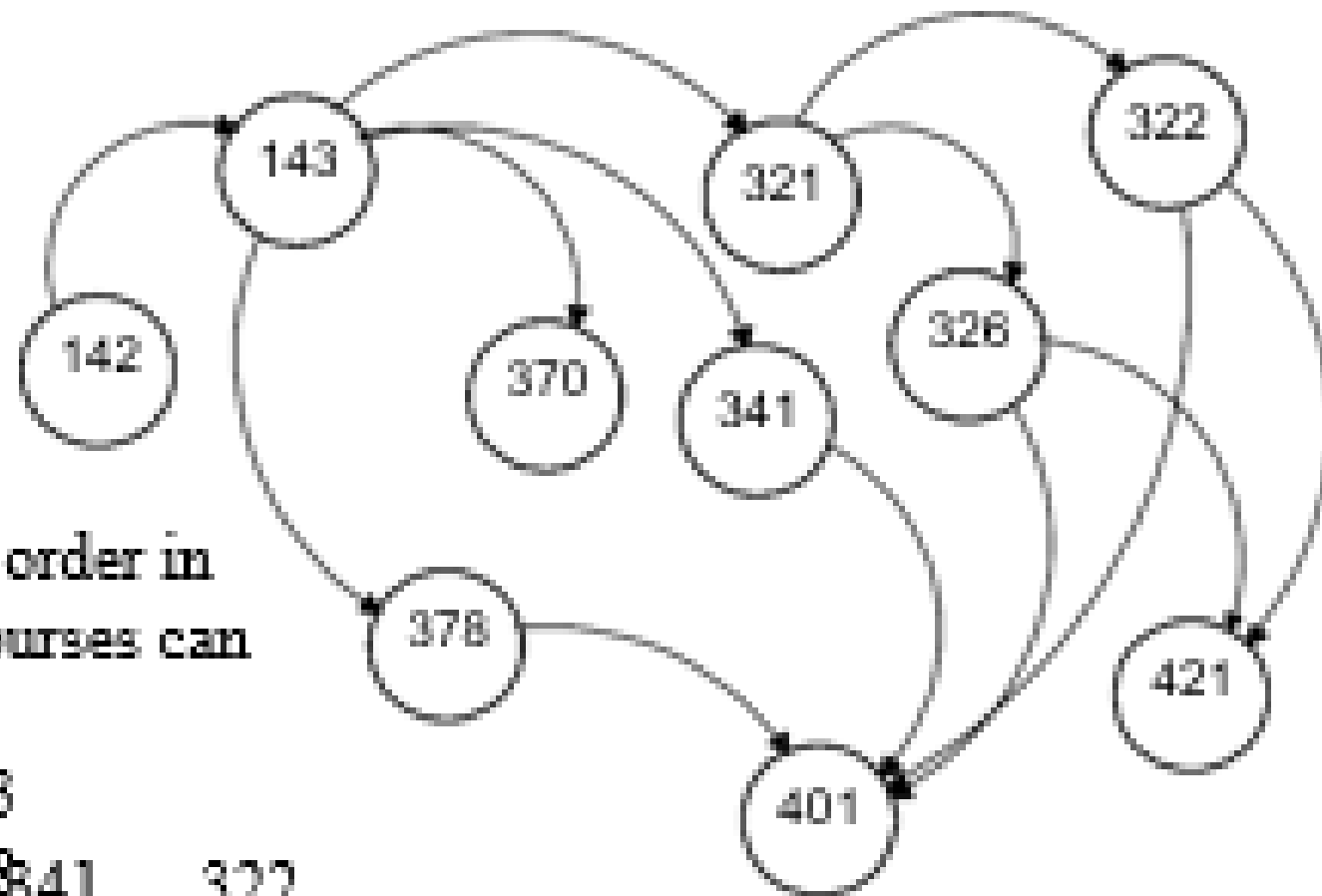
- $T(n) = T(n/2) + c$, $T(1) = d$, where d is a constant
- ▮ $T(n) = T(n/2) + c = T(n/2^2) + 2c = \dots = T(n/2^k) + kc$
- ▮ $= kc + d$ (Assuming $n=2^k$, which implies $k = \log n$)
- ▮ $= O(\log n)$



Analysis of Binary Search

- This is VERY fast: e.g., $T(2^{60}) = 60$
- Sequential search will take $O(2^{60})$
- Optimal for searching a sorted array
- Limitations: must be a sorted array

Topological Sort



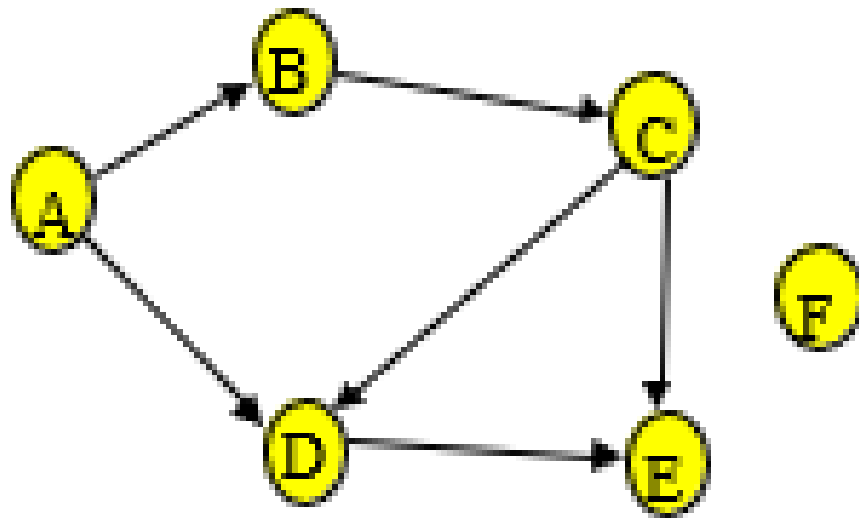
Problem: Find an order in which all these courses can be taken.

Example: 142 143

370	321	378	341	322
326	421	401		

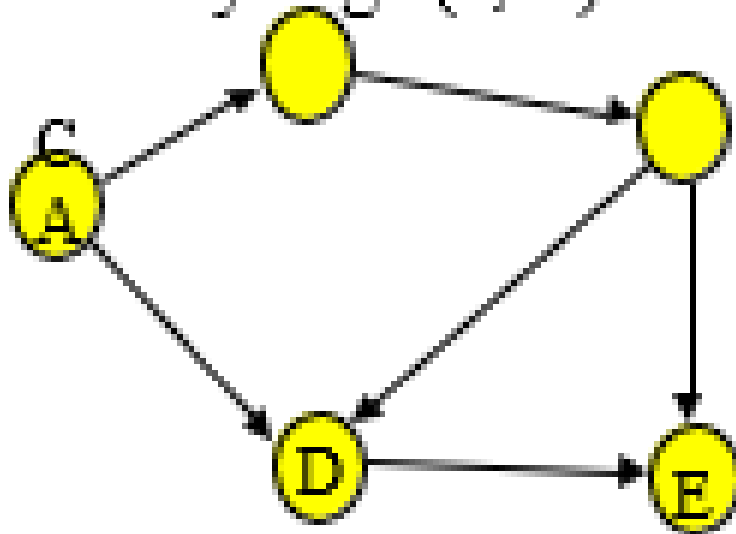
Topological Sort Definition

Topological sorting problem: given digraph $G = (V, E)$,
find a linear ordering of vertices such that:
for all edges (v, w) in E , v precedes w in the ordering



Topological Sort

Topological sorting problem: given digraph $G = (V, E)$, find a linear ordering of vertices such that:
for any edge (v, w) in E , v precedes w in the ordering

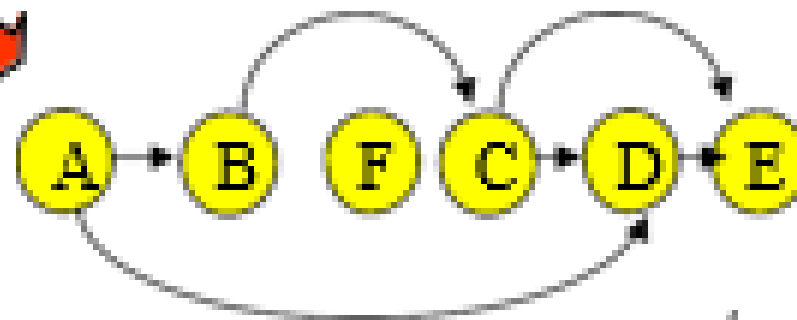


B

F



*Any linear ordering in which
all the arrows go to the right
is a valid solution*

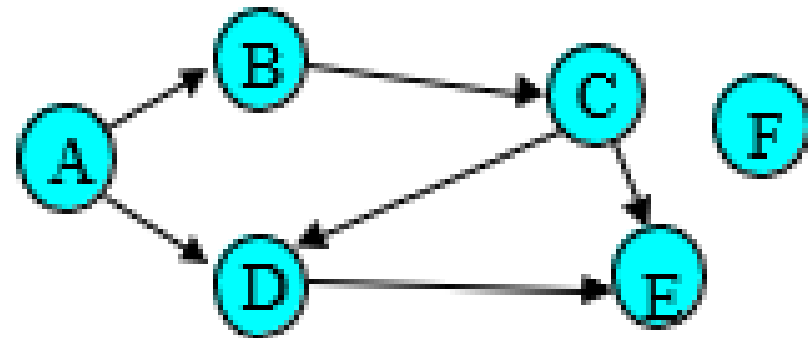


Topological Sort

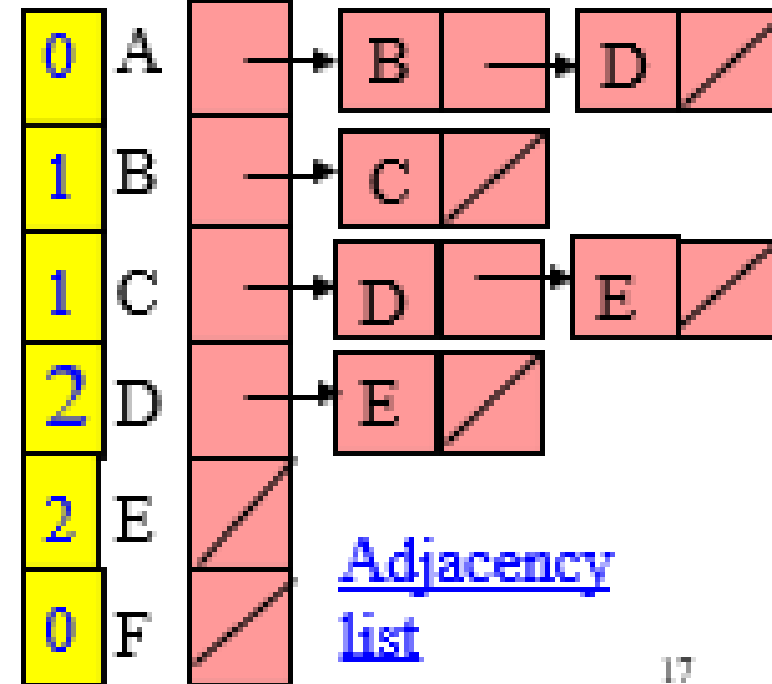
- Topological sort can only be applied to **Directed Acyclic Graphs (DAGs)**. A DAG is a graph that has directed edges with no cycles.
- If the digraph contains cycles, it is not possible to find a valid topological ordering.
- Topological sorting is commonly used in various applications, such as task scheduling, dependency resolution, and in finding a linear order of events in a system.

Key idea: Initialize and maintain a *queue* (or *stack*) of vertices with In-Degree 0

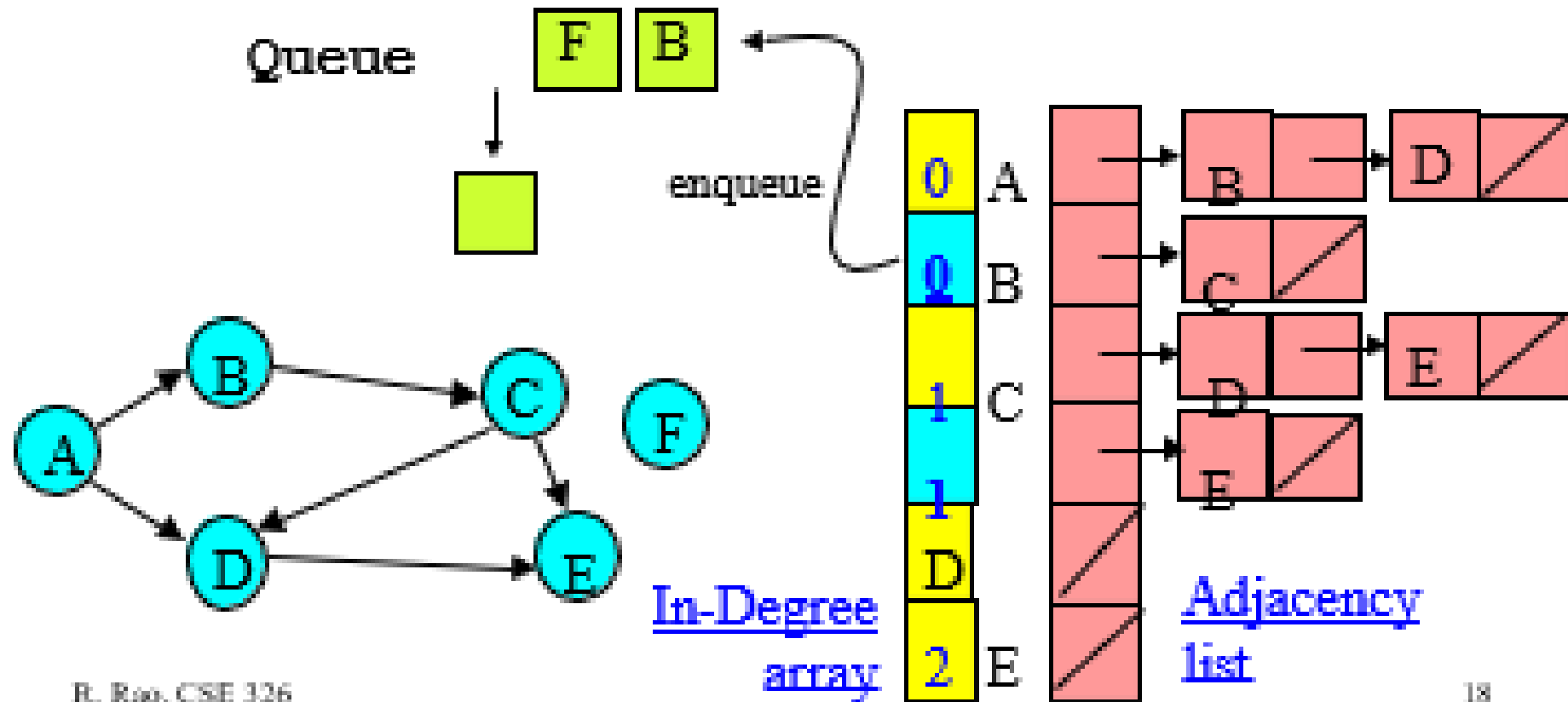
Queue A F



In-Degree
array

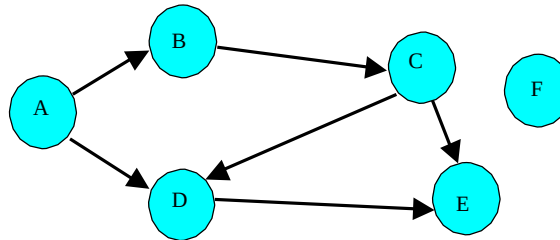


After each vertex is output, when updating In-Degree array, *enqueue any vertex whose In-Degree has become zero*



Topological Sort Algorithm #2

1. Store each vertex's **In-Degree** in an array
2. Initialize a queue with all in-degree zero vertices
3. While there are vertices remaining in the queue:
 - ⇒ Dequeue and output a vertex
 - ⇒ Reduce In-Degree of all vertices adjacent to it by 1
 - ⇒ Enqueue any of these vertices whose In-Degree became zero



Sort this digraph!

Topological Sort Algorithm: Analysis

For input graph $G = (V, E)$, Run Time = ?

Break down into total time to: Initialize In-Degree array:

$O(|E|)$

Initialize Queue with In-Degree 0 vertices: $O(|V|)$

Dequeue and output vertex:

$|V|$ vertices, each takes only $O(1)$ to dequeue and output.

Total time = $O(|V|)$

Reduce In-Degree of all vertices adjacent to a vertex and

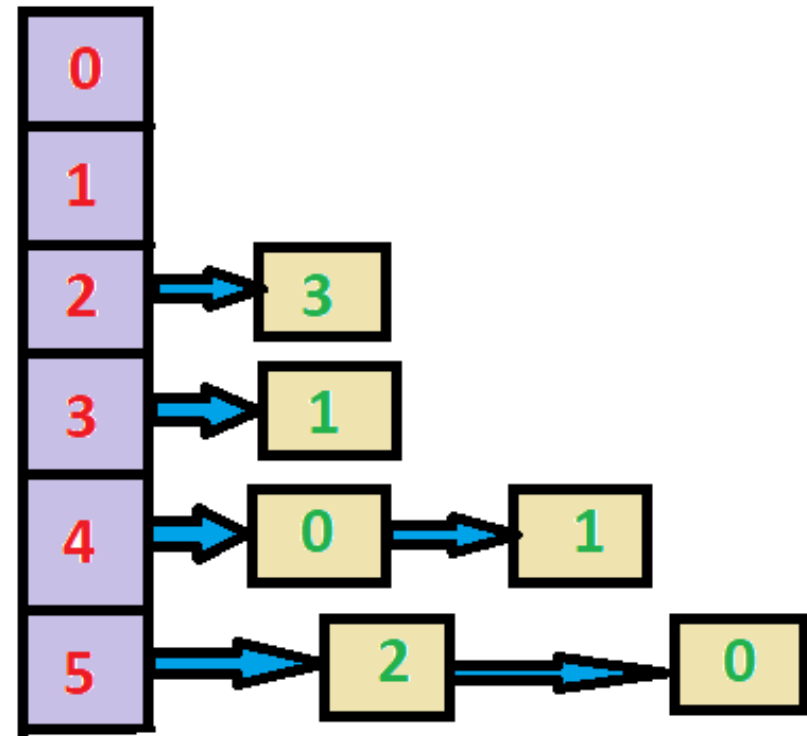
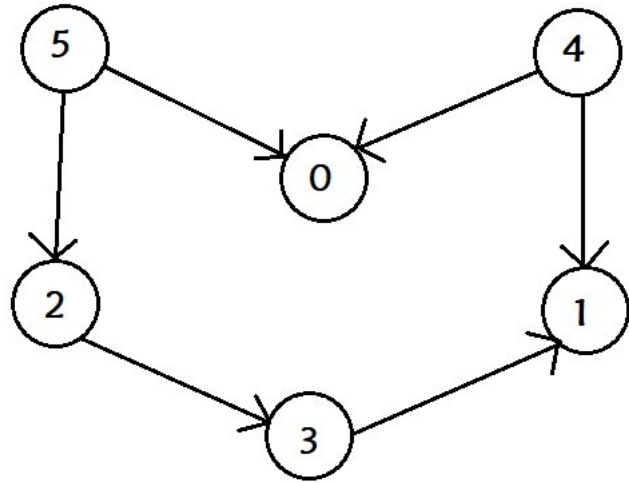
Enqueue any In-Degree 0 vertices:

$O(|E|)$

Total time = **$O(|V| + |E|)$** **Linear running time!**

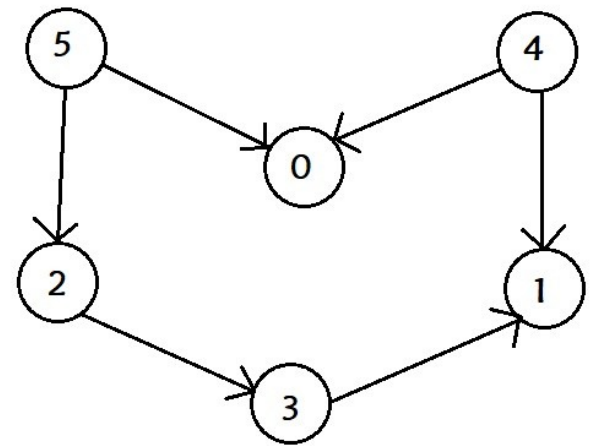
Topological Sort DFS Method

- **Step 1:** Create a temporary stack.
 - **Step 2:** Recursively call topological sorting for all its adjacent vertices, then push it to the stack (when all adjacent vertices are on stack). Note this step is same as Depth First Search in a recursive way.
 - **Step 3:** Finally, print the contents of stack.
-
- Note: A vertex is pushed to stack only when all of its adjacent vertices (and their adjacent vertices and so on) are already in stack.



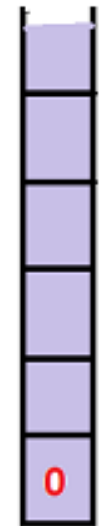
VISITED[] =

F	F	F	F	F	F
0	1	2	3	4	5



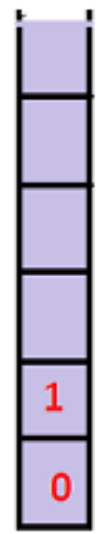
i=0

T	F	F	F	F	F
0	1	2	3	4	5



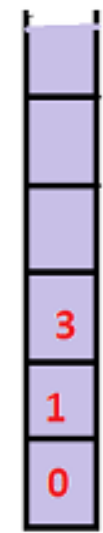
i=1

T	T	F	F	F	F
0	1	2	3	4	5



i=2

T	T	T	F	.	.
0	1	2	3	4	5



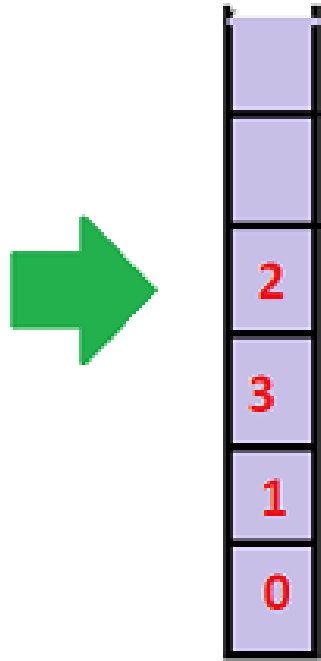
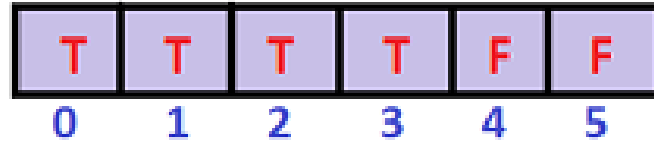
T	T	T	T	F	F
0	1	2	3	4	5

Go to node 0, it has no outgoing edges(i.e. no successors) so push node 0 into the stack and mark it visited.

Go to node 1, it has no outgoing edges(i.e. no successors) so push node 1 into the stack and mark it visited.

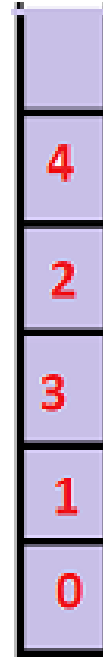
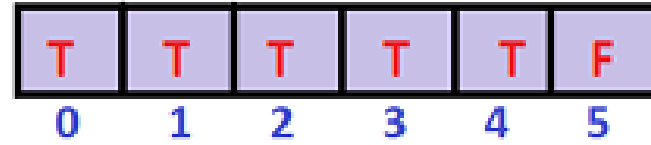
Go to node 2, process all the adjacent nodes(includes node) and mark node 2 visited

i=3



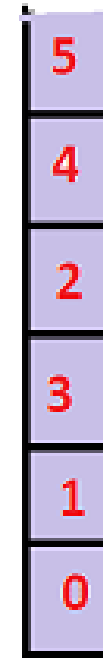
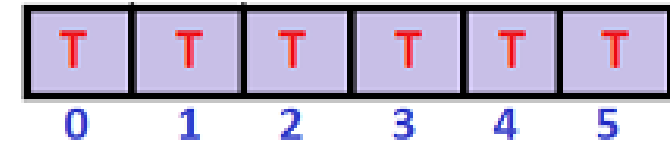
Node 3 is already visited so continue with next iteration

i=4

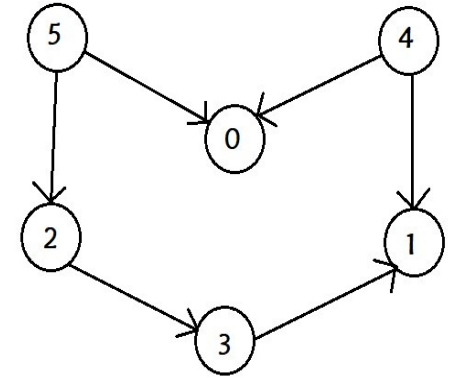


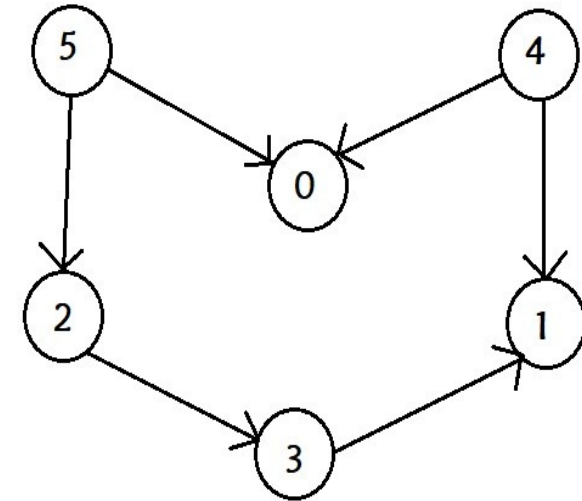
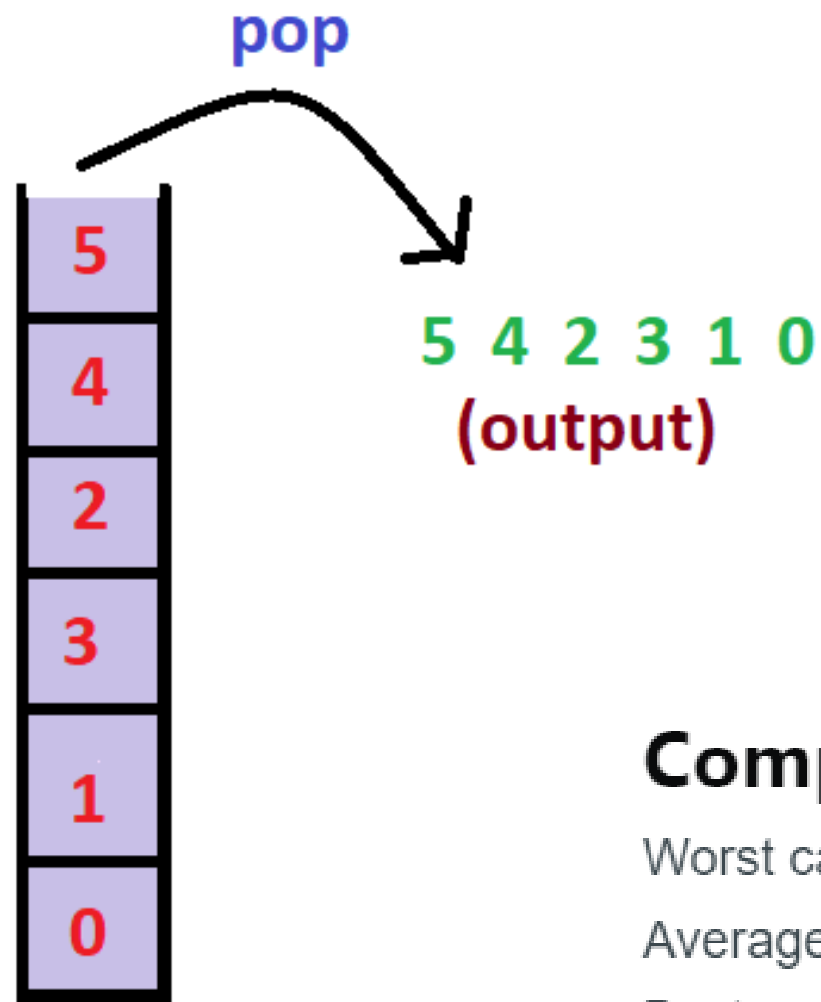
Go to node 4, all its adjacent nodes are already visited so push node 4 into the stack and mark it visited.

i=5



Go to node 5, all its adjacent nodes are already visited so push node 5 into the stack and mark it visited.





Complexity

Worst case time complexity: $\theta(|V| + |E|)$

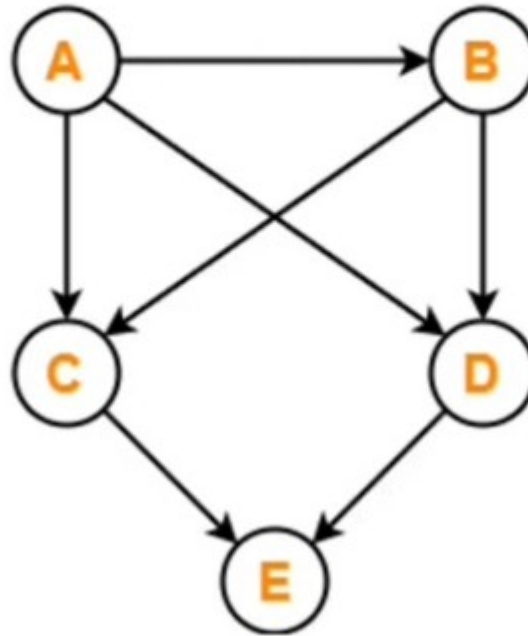
Average case time complexity: $\theta(|V| + |E|)$

Best case time complexity: $\theta(|V| + |E|)$

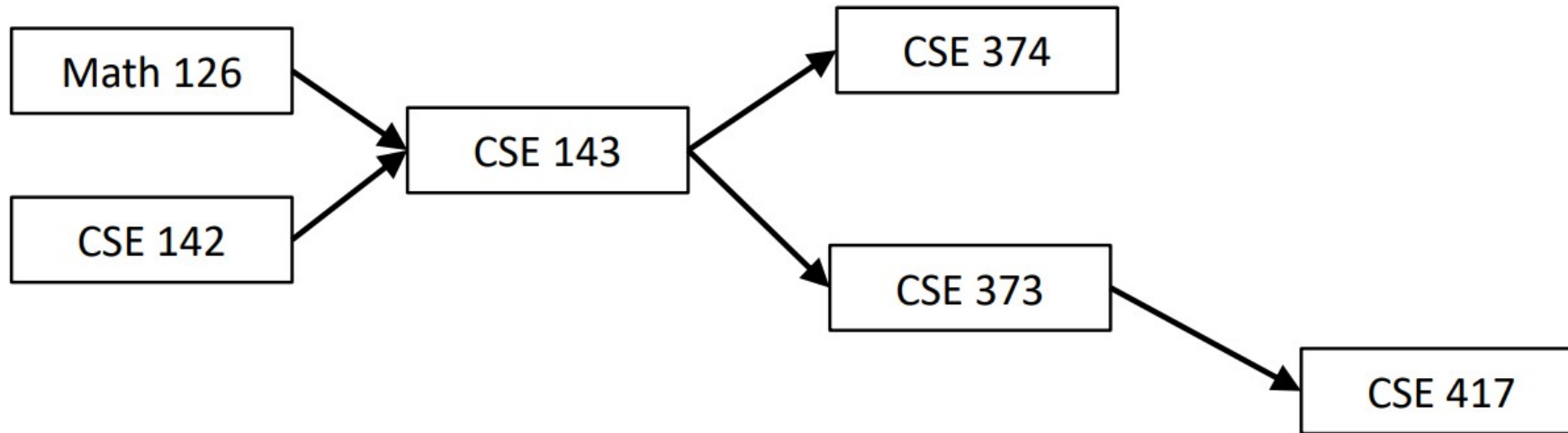
Space complexity: $\theta(|V|)$

Problem

Problem 01: Find the topological ordering for the given graph by using Depth First Search.



- Prerequisite for CSE 143 course is Math 126 and CSE 142
- Prerequisite for CSE 373 and CSE 374 is CSE 143
- Prerequisite for CSE 417 is CSE 373



Activity	A	B	C	D	G	E	F	L	I	J	M	K
Duration, days	2	3	4	5	3	3	2	1	4	3	5	6
Following activity	C,D	G	E	F	L	I	J	M	K	K	K	---

activity	A	B	C	D	E	F	H	K	G	I	J	M
Duration, days	3	4	5	7	5	4	3	3	6	2	11	6
Following activity	D,E	E	F	H	K	K,G	I,J	M	M	M	M	---

1. Draw activity network for the given table and sort the activity as per their topological ordering.

Activity	Duration (week)	Predecessors
A	1	–
B	2	–
C	2	A
D	4	A
E	1	B
F	2	C, D
G	3	E
H	1	G
I	4	G
J	1	F
K	3	J, H
L	4	I
M	1	K, L